

I. Introduction

According to WHO, over 5% of the world's population – or 430 million people – require rehabilitation to address their disabling hearing loss (including 34 million children). It is estimated that by 2050 over 700 million people – or 1 in every 10 people – will have disabling hearing loss. People who are deaf usually suffer from mute so they cannot use their voice to express their opinion. Therefore, using sign language is a great way for people who have that kind of disability to communicate with each other or with people around the world. There are somewhere between 138 and 300 different types of sign language used around the globe today. Interestingly, most countries that share the same spoken language do not necessarily have the same sign language as each other. English for example, has three varieties: American Sign Language (ASL), British Sign Language (BSL) and Australian Sign Language (Auslan). In this research, we mainly focus on America Sign Language because it is the most widely used sign language around the world. Signers are spread across the USA and Canada, as well as parts of Mexico, Africa, and Asia. It developed when French Sign Language (FSL) was brought over to the USA in 1817 by Thomas Gallaudet and Laurent Clerc. There, it mixed with local indigenous languages to create what is now called ASL. Besides that, there are also a lot of papers that work which ASL so there will be many resources for me to do the research.

II. Goals

The target that I aim is creating an efficiency model that help to recognize American Sign Language with high accuracy and have a large dataset. Therefore, it can then be used to apply for making applications to help people with disabilities to have another way to communicate with people who don't understand sign language.

III. Requirements

1. Functional Requirements

- User should be able to log in to the web to track their learning progress.
- User should be able to register a new account with their email and password; a confirmed email should be sent to user for verification.
- Users can update their personal information on the web.
- User should be able to search for word in the dictionary web by 2 ways: search by word or search by sign language (The web will display a camera for user to make the sign).
- Users should be able to find words according to topics that are given by the web.
- The web should have a webpage that record user sign and output text that follow American Sign Language rules.
- Users with registered accounts can add favorite words to their favorite list.
- Users with registered accounts can make new sign language words and send that word to admin as a suggestion.
- In a word detail, there should be some related words as recommendations.
- The web should provide flashcards for users to learn and practice new words; every word must be learned 4 times to make the progress 100%.

- The web should display user statistics about how many words are processed to 100%, how many words suggesting admin are accepted, and how many times learning.
- The web should have a page for users to contact admin.
- Admin should be able to ban users that violate the policy of the web.
- Admin can check and decide to take user suggestions about the new word to be the word for the dictionary web.
- Admin should be able to see how many users access the web through links, Facebook, Instagram, and Twitter.
- Admin should be able to customize some of the webpage components.
- Storage function: the storage feature is one of the most important features of the system, which is the basis for implementing the system's features. To ensure effective management, the system needs to store the following information:
 - User information: id, username, email, password, fullname, date of birth, gender, status, created time, updated time.
 - Word information: idWord, name, description, example, video, relative word, viewers, liked, contributor, topic.
 - Topic information: idTopic, name, number of words, word
 - Favorite word information: idUser, favorite, created time, updated time
 - New word information: idword, idUser, name, description, example, video, relative word, created time, updated time, status

- Learned word information: id, idUser, idWord, process
- Access Statistic information: id, access by web, access by facebook, access by twitter, access by Instagram
- Contact us information: id, first name, last name, email, message, created time, status

2. Non-functional Requirements

- Performance:
 - Response Time: The web application should respond to user actions within a certain timeframe, under 2 seconds for common tasks.
 - Throughput: The system should be able to handle a certain number of concurrent users or transactions, supporting 100 simultaneous users.
 - Resource Utilization: The application should not consume excessive server resources like CPU, memory, or bandwidth.
- Usability:
 - User Interface Responsiveness: The UI should be responsive to user interactions, providing feedback promptly.
 - Consistency: User interface elements and interactions should be consistent throughout the application.
- Reliability:
 - Availability: The system should be available for use during specified hours, with minimal downtime
 - Fault Tolerance: The application should be able to gracefully handle errors, failures, and exceptions without crashing or losing Data.

- Disaster Recovery: Plans and procedures should be in place to recover data and restore services in case of catastrophic failures.

- Security:

- Authentication: Users should be able to securely authenticate themselves, preferably through secure protocols

- Authorization: Access to resources and functionality should be appropriately restricted based on user roles and permissions.

- Data Protection: Sensitive data should be encrypted during transmission and storage to prevent unauthorized access.

- Scalability:

- Horizontal Scalability: The application should be able to scale out by adding more servers or instances to handle increased load.

- Vertical Scalability: The application should be able to scale up by utilizing more powerful hardware resources as needed.

- Load Balancing: Traffic should be distributed evenly across servers to prevent any single point of failure and ensure optimal performance.

- Maintainability:

- Modifiability: The codebase should be designed in a way that allows for easy modification and extension without causing unintended side effects.

- Testability: The application should be designed with automated testing in mind to facilitate continuous integration and deployment.

- Documentation: Clear and comprehensive documentation should be provided for developers, administrators, and users.

- Compliance:

- Regulatory Compliance: The application should comply with relevant laws, regulations, and industry standards.

- Standards Compliance: The application should adhere to relevant web standards and best practices to ensure interoperability and compatibility.

- Interoperability:

- Integration with third-party systems: The application should be able to integrate seamlessly with other systems or services through APIs or standard protocols.

- Cross-browser compatibility: The application should function correctly and consistently across different web browsers

- User Experience (UX):

- Responsiveness: Ensuring that the application feels responsive and interactive, with smooth transitions and minimal latency.

- Error Handling: Presenting informative error messages and guiding users on how to resolve issues when errors occur.

- Auditability and Logging:

- Logging: Capturing and logging relevant events, actions, and errors for auditing, troubleshooting, and analysis purposes.

- Audit Trails: Maintaining a record of user activities and system events for compliance, security, and accountability purposes.

- Documentation and Training:

- User Documentation: Providing comprehensive documentation and user guides to help users understand how to use the application effectively.

- Training Materials: Developing training materials and resources for administrators, supporting staff, and end users to facilitate adoption and usage of the application.

IV. Functionality

- Sign language detection
- Learning process tracking for user
- Manage users
- Manage words
- Mini practices for users
- Add favorite words
- Create own words and make suggestions

V. Tech stack

- **Dataset:** Kaggle
- **Object Detection:** YOLOv5, Python, OpenCV.
- **Model Architecture:** RNN, CNN
- **Training Tool:** Google Colab, Roboflow
- **IDE:** Visual Studio
- **Language:** C#, Javascript
- **Framework:** ReactJS, .NET
- **Database:** MongoDB

VI. Schedule

1. Phase 1 (Week 1 - 5)

- Planning and architecture:

- System architecture diagram
 - Set up repo, project management board
- Data pipeline
 - Build preprocessing pipeline
 - Data augmentation strategy, train/val/test split
 - Start backend scaffolding in parallel: DB schema, auth requirement
- Model baseline:
 - Build baseline model
 - Get an end-to-end training loop running, even with mediocre accuracy
 - Backend: core API skeleton
- Model iteration:
 - Train multiple architecture/hyperparameter variants
 - Track experiments (accuracy, latency, confusion matrix per sign class)
 - Backend: user history/logging requirement, DB persistence layer
 - Frontend: basic scaffolding, routing, auth screens
- Model finalization:
 - Pick best model, do final tuning
 - Optimize for inference
 - Wrap model in an inference service/API
 - Frontend: webcam capture component started

2. Phase 2 (Week 6 - 10)

- Core Integration:
 - Connect frontend webcam feed to backend, do model inference and then prediction returned
 - Detect a sign and show text
- Feature buildout:
 - Gesture-to-text display polish
 - Text-to-speech output requirement
 - Confidence threshold / "no sign detected" handling
- Remaining requirements and hardening

- Testing, deployment, and docs:
 - End-to-end QA against all requirements
 - Deployment
 - Documentation: architecture, API, model card (accuracy, limitations, dataset used)
 - Demo prep / final presentation materials

VII. Testing plan

1. Unit Testing

- Sign language detection: test model inference functions, input preprocessing (frame extraction, normalization), confidence threshold logic
- Learning tracking: test progress calculation, streak logic, score aggregation functions
- User management: test CRUD operations, password hashing, role/permission checks
- Word management: test CRUD operations, validation (duplicate words, required fields)
- Mini practices: test question generation logic, scoring algorithms, randomization
- Favorites: test add/remove/toggle logic, duplicate prevention
- Word suggestions: test submission validation, status transitions (pending/approved/rejected)

2. Integration Testing

- API endpoints ↔ database (all modules: users, words, favorites, practices, suggestions)

- Sign detection model ↔ backend API (video/image upload → prediction → response)
- Learning tracker ↔ practice results (does completing a practice update progress correctly?)
- Authentication middleware across all protected routes
- Third-party services if used (cloud storage for videos/images, notification services)

3. ML Model-Specific Testing

- Accuracy/precision/recall on labeled test dataset (per sign class)
- Edge cases: poor lighting, partial hand visibility, different skin tones, backgrounds, camera angles
- Latency testing (real-time detection responsiveness)
- Confusion matrix review for commonly misclassified signs
- Test on varied devices (webcam quality, mobile camera vs desktop)
- Model versioning tests (rollback if new model underperforms)

4. Functional / System Testing

- End-to-end user flows: register → learn → practice → track progress → favorite words → suggest new word
- Admin flows: manage users (ban/edit/delete), manage words (approve/reject/edit), review suggestions
- Cross-browser and cross-device testing (especially camera/mic permissions)

5. Non-Functional Testing

- Performance/load testing

- Security testing: auth token handling, input sanitization, file upload validation (video/image), rate limiting on detection API
- Usability testing with real target users (including accessibility for hearing-impaired users, screen reader compatibility, captions, etc.)
- Data privacy testing (especially if storing video/webcam data, ensure compliance, e.g., consent, retention policy)

6. User Acceptance Testing (UAT)

- Beta test with a small group representing target users
- Collect feedback specifically on detection accuracy and learning flow intuitiveness

VIII. Deployment plan

1. Environment Setup

- Separate Dev, Staging, Production environments
- Environment-specific config (API keys, DB connections, model versions) via environment variables/secrets manager

2. CI/CD Pipeline

- Automated build + unit/integration tests on every PR/commit
- Staging deployment auto-triggered on merge to main/develop branch
- Manual approval gate before production deployment
- Automated rollback mechanism if health checks fail post-deploy

3. Infrastructure

- Backend: containerize with Docker; orchestrate via Kubernetes or a managed service (ECS, Cloud Run, etc.) if scaling is a concern
- ML model serving consider separate microservice/endpoint for detection (e.g., TensorFlow Serving, TorchServe, or a lightweight FastAPI wrapper) so it can scale independently of the main app

- Database: managed DB service (RDS, Cloud SQL) with automated backups
- Media storage: object storage (S3 or equivalent) for word images/videos and user practice recordings if applicable
- CDN for static assets (word images, tutorial videos)

4. Deployment Strategy

- Use blue-green or canary deployment for the detection service specifically, since model updates carry accuracy risk
- Feature flags for new features (e.g., word suggestion feature) to enable gradual rollout
- Database migrations run as a separate, reversible step before app deployment

5. Monitoring & Observability

- Application logs and error tracking (Sentry or similar)
- Model performance monitoring in production (detection accuracy drift, latency, failure rate)
- Uptime monitoring for API and detection service
- User analytics on feature usage (practice completion rate, favorite usage, suggestion submission rate)

6. Post-Deployment

- Smoke tests immediately after deploying (core flows: login, detection, practice)
- Rollback plan documented and tested
- Scheduled model retraining/update pipeline if you plan to improve detection accuracy over time

IX. Conclusion

A Dictionary is somewhat that is very familiar to us, especially when we are learning a new language. When it comes to creating a dictionary website that helps people to learn sign language, it is an interesting but also challenging task. It is difficult for people to learn and check their skills on the same website. Therefore,

with the sign language web project, our team wants to integrate everything into one, giving users the ability to learn about new sign language and check their skill using sign language detection. We have built 2 sites. One for users who want to learn and test their skills, and the other one is for administrators who control and operate the system. Those tasks took us a lot of time, but it is worth for us.